

ADR-005 — Loopback transport와 sidecar 보안 계약

항목	값
상태	Accepted
결정일	2026-08-20
적용 결정	D-14
API major version	v1
schema version	3

1. 경계와 책임

React/Tauri renderer와 Python backend는 별도 process다. Communication Diagram의 논리 호출은 loopback HTTP command/query와 WebSocket event로 운반한다.

- **BackendUiAdapter**는 wire 직렬화, 인증 header, timeout, reconnect와 UI contract mapping만 담당한다.
- transport route는 요청 검증 후 기존 Controller Operation을 호출한다.
- adapter와 route는 REGIME 지원 여부, 주문 Guard, stop branch와 Performance 공식을 다시 판단하지 않는다.
- domain object를 그대로 JSON으로 내보내지 않고 versioned DTO로 변환한다.
- 금융 Decimal은 JSON string, 모든 backend 시각은 UTC RFC 3339 Z다.

2. Bind와 process handshake

- backend는 IPv4 127.0.0.1의 OS 할당 random port에만 bind한다.
- 0.0.0.0, LAN 주소와 public interface bind를 금지한다.
- 한 app launch마다 Tauri가 CSPRNG 32 bytes의 session token을 만들고 base64url no-padding 문자열로 표현한다. 재실행 시 이전 token을 재사용하지 않는다.
- token은 command-line argument, URL, source, .env, 일반 environment variable과 log로 전달하지 않는다. Tauri가 만든 anonymous inherited pipe/file descriptor로 backend에 한 번 전달한다.
- backend ready 신호에는 port, session_id, positive runtime_pid, canonical process_start_id, schema_version만 포함하고 token을 출력하지 않는다.
- native lifecycle은 직접 spawn한 launcher child PID/handle과 READY의 actual Python runtime_pid/process_start_id를 서로 다른 identity로 보존한다. Python app-data .backend-runtime.lock의 durable artifact는 schema_version, runtime_pid, process_start_id, owner_state 네 field만 포함하며 port-parent PID-token을 기록하지 않는다.
- Tauri는 현재 backend 실행의 connection descriptor를 native 메모리에 보존한다. get_backend_connection_descriptor는 main 창과 실행 환경의 정확한 화면 origin을 확인한 뒤 live adapter에 전달한다. 화면 새로고침과 연결 복구는 같은 backend session으로 재연결하며, process 전체에서 한 번만 소비하여 복구 버튼을 막지 않는다.
- token은 native와 adapter closure 메모리 밖의 local/session storage, IndexedDB, Redux/XState snapshot, URL, error message와 console에 넣지 않는다. native IPC 응답 복사본은 직렬화 후 zeroize하고, 원본도 backend 종료 시 즉시 제거-zeroize한다.

token은 “한 요청에 한 번 쓰는 token”이 아니라 한 process launch에서만 유효한 session token이라는 의미로 one-time이다. backend 재시작 시 기존 token과 session ID는 즉시 폐기한다.

3. HTTP 인증과 공통 응답

모든 /v1/* 요청은 다음 header를 요구한다.

```
Authorization: Bearer <session-token>
X-Request-Id: <UUID>
Content-Type: application/json      # body가 있는 요청
```

POST/PATCH command는 추가로 Idempotency-Key를 요구한다. token 비교는 constant-time으로 수행한다. 인증 실패는 body에 세부 이유를 노출하지 않고 401 AUTHENTICATION_REQUIRED를 반환한다.

성공 응답은 다음 envelope를 사용한다.

```
{
  "schema_version": 3,
  "request_id": "uuid",
  "ok": true,
  "data": {}
}
```

실패 응답은 다음 envelope를 사용한다.

```
{
  "schema_version": 3,
  "request_id": "uuid",
  "ok": false,
  "error": {
    "code": "UNSUPPORTED_TRADING_LOGIC",
  }
}
```

```

    "message": "User-safe message",
    "retryable": false,
    "details": {}
  }
}

```

`details`에는 credential, token, raw Binance response, filesystem secret path와 stack trace를 넣지 않는다. 같은 `Idempotency-Key`와 같은 body의 재요청은 저장한 동일 결과를 반환하고, 같은 key에 다른 body가 오면 `409 IDEMPOTENCY_CONFLICT`다.

4. Endpoint 계약

Method	Path	Request 핵심	Response 핵심	Controller owner
GET	/v1/health	없음	process/session/schema/ready, secret 없음	bootstrap 함수
GET	/v1/snapshot	없음	일관된 전체 snapshot + <code>session_id</code> + <code>last_sequence</code>	bootstrap/read model 조합 함수
GET	/v1/trades	<code>period</code> , <code>side</code> query	rows, row count; summary 범위는 ADR-004	<code>TradeHistoryController.getTradeDetails</code>
POST	/v1/regime/selecion	<code>regime_type</code> , <code>expected_version</code>	selected/support 상태와 새 version	<code>RegimeController.setRegimeType</code>
POST	/v1/trading/start	<code>expected_version</code>	command status, session/version	<code>TradingController.startTrading</code>
POST	/v1/trading/stop	<code>expected_version</code>	STOPPING/TERMINATED/reconciliation 상태	<code>TradingController.stopTrading</code>
POST	/v1/trading/recovered-position/liquidate	<code>expected_version</code>	복구 Position 청산의 STOPPING/TERMINATED/reconciliation 상태	<code>TradingController.liquidateRecoveredPositi</code>
PATCH	/v1/trading/manual-kill	exact boolean <code>active</code> , <code>expected_version</code>	kill 상태, policy provenance와 새 risk control version	<code>TradingController.setManualKill</code>
PATCH	/v1/trading/split-ratios	Decimal string <code>scale_in</code> , <code>scale_out</code> , version	적용 값과 새 version	<code>TradingContext</code> 를 조정하는 <code>TradingController</code>
POST	/v1/csv-exports	ADR-004의 option DTO	path, row count 또는 typed failure	<code>TradeHistoryController.exportCSV</code>
POST	/v1/shutdown	<code>expected_version</code>	accepted/blocked와 안전 상태	bootstrap lifecycle 함수가 기존 stop/flush Operation 조정
WS	/v1/events	첫 frame 인증과 <code>after_sequence</code>	versioned backend event stream	event stream transport 함수

GET /v1/trades의 날짜-side validation과 CSV의 업무 validation은 Controller/domain value object가 최종 수행한다. route의 schema validation은 type/shape/size 제한만 담당한다.

schema version 2의 /v1/snapshot.trading.logic_coverage는 canonical 순서의 `type0~type4`를 정확히 한 번씩 포함한다. 각 행은 `support_status`와 `start_guard`를 가지며 UI는 누락·중복·알 수 없는 조합을 fail closed한다. 이는 시작 안전성에 필요한 필수 필드이므로 해당 필드가 없던 schema version 1과 구분한다.

schema version 3은 Phase 13의 `risk_policy_availability`, configured/session policy version, `risk_control_version`, `manual_kill_active`, 최근 typed risk 차단 사유와 `process_ownership_ambiguous`를 trading snapshot의 필수 필드로 추가한다. UI는 backend보다 앞서 성공이나 안전 상태를 추정하지 않고 이 authoritative publication만 표시한다. 이 필드가 없는 schema version 2와 process identity가 없는 READY descriptor는 fail closed한다.

HTTP status 기본 매핑은 다음과 같다.

Status	의미
200	query 또는 idempotent no-op 성공
201	동기 CSV 파일 publication 완료와 actual path/row-count receipt
202	장기 stop/shutdown command 수락
400	malformed DTO 또는 지원하지 않는 schema
401	token 없음/불일치
403	origin/host/mode/live gate 거부
409	version, active session, idempotency 또는 destination 충돌
422	domain validation 실패 또는 미지원 REGIME
503	backend not ready, offline 또는 reconciliation lock

CSV는 별도 job/status endpoint가 없는 `exportCSV(): CSVExportResult` 계약이므로 Phase 11에서 파일의 fsync-atomic rename이 끝난 뒤 201과 actual receipt를 동기 로 반환한다. job ID만 받고 나중에 결과를 조회하는 비동기 202 export는 별도 ADR과 polling/event 계약 없이는 사용하지 않는다.

5. Event envelope

인증 뒤의 모든 server event는 다음 필드를 가진다.

```
{
  "schema_version": 3,
  "session_id": "uuid",
  "event_id": "uuid",
  "sequence": 42,
  "occurred_at": "2026-08-20T11:40:00.123456Z",
  "type": "ORDER_EXECUTED",
  "aggregate_version": 17,
  "correlation_id": "command-or-decision-id",
  "payload": {}
}
```

- `sequence`는 한 backend session에서 1부터 시작하는 unsigned 64-bit monotonic 정수다.
- 한 `sequence`는 정확히 한 envelope에만 사용한다.
- `event_id` 중복은 UI에서 no-op이고, 더 작은/equal `sequence`도 다시 적용하지 않는다.
- `aggregate_version`이 없는 event는 `null`이다.
- 알 수 없는 `type`은 연결을 죽이지 않고 기록·무시할 수 있지만, 알 수 없는 major `schema_version`은 `UNSUPPORTED_SCHEMA_VERSION`으로 fail closed한다.
- payload의 Decimal과 ID는 string이며 raw credential/Binance response를 넣지 않는다.
- 현재 process가 모르는 `bat-` 주문 event는 application callback 실패로 숨기지 않는다. backend가 command gate를 닫은 authoritative trading lifecycle snapshot을 즉시 event stream에 게시하고, 별도 recovery worker가 REST 재조정을 수행한다.
- account stream 복구 성공은 두 REST snapshot의 authoritative Account를 먼저 게시하고, 이어 다시 열린 command gate와 trading lifecycle을 게시한다. 재조정 commit 부터 이 publication까지 같은 application RLock을 유지하며, publication 실패는 backend-only 주문 재개를 허용하지 않고 영구 reconciliation gate로 닫는다.

6. WebSocket 인증과 reconnect

브라우저 WebSocket은 임의 Authorization header를 안정적으로 넣을 수 없으므로 token을 URL query나 subprotocol에 넣지 않는다.

- client가 `/v1/events`에 연결한다.
- 2초 안에 첫 frame으로 아래 인증 메시지를 보낸다.

```
{
  "schema_version": 3,
  "type": "AUTHENTICATE",
  "token": "session-token",
  "after_sequence": 41
}
```

- 인증 전 server는 application event를 보내지 않는다.
- token 불일치, 두 번째 인증 시도 또는 timeout이면 policy violation으로 연결을 닫는다.
- 인증 frame 자체와 token은 log/trace에 기록하지 않는다.

backend는 최근 10,000개 또는 15분 중 먼저 도달하는 범위의 event replay buffer를 유지한다.

- 최초 연결: UI가 `/v1/snapshot`을 받고 `last_sequence`를 `after_sequence`로 연결한다.
- 정상 reconnect: buffer에 `sequence`가 있으면 `after_sequence + 1`부터 순서대로 replay한 뒤 live event를 보낸다.
- buffer gap, 새 `session_id`, sequence 역행: server가 `RESYNC_REQUIRED`를 보내고 UI는 cache를 이어 붙이지 않고 새 snapshot부터 다시 시작한다.
- snapshot은 생성 시점까지의 일관된 상태와 그 상태에 포함된 `last_sequence`를 원자적으로 반환한다.

7. Host, Origin, CORS와 resource 제한

- `Host`는 실제 `127.0.0.1:<assigned-port>`와 정확히 일치해야 한다.
- production Origin allowlist는 Tauri app의 정확한 origin만 포함한다. 개발 origin은 dev mode 설정에 명시한 정확한 localhost port만 추가한다.
- wildcard CORS, credentials 기반 cross-origin cookie와 browser cookie 인증을 사용하지 않는다.
- HTTP body, query page size와 WebSocket frame에 명시적 최대 크기를 둔다. 기본 상한은 command body 1 MiB, event frame 1 MiB, trade page 1,000 rows다.
- access log는 method, route template, status, duration, request ID만 기록하고 header, query의 민감 값, body와 token을 기록하지 않는다.
- CSP는 배정된 loopback origin과 필요한 Tauri capability만 허용한다.

8. 오류와 shutdown

- backend가 ready 전이면 command를 503 `BACKEND_NOT_READY`로 거부한다.
- sidecar crash 또는 session ID 변경 시 adapter는 신규 command를 막고 UI를 offline으로 전환한다.
- runtime ownership artifact는 startup에 `ACTIVE`, parent control FD EOF에 `ORPHANED`, 정상 종료에 `RELEASED`를 fsync한다. OS advisory lock이 비어 있어도 `ACTIVE/ORPHANED`를 새 owner가 덮어쓰지 않으며, operator reconciliation 없이 sidecar를 자동 relaunch하지 않는다.
- shutdown은 ADR-003의 open order/Position 안전 조건을 먼저 검사한다. 안전하지 않으면 409 `SHUTDOWN_BLOCKED_BY_OPEN_EXPOSURE`를 반환한다.
- 정상 shutdown은 신규 command 차단 → trading stop 완료 → history flush/fsync → stream close → process exit 순서를 따른다.

9. 검증 의무

- ☒ endpoint, 공통 응답과 event envelope schema가 확정되었다.
- ☒ loopback bind, per-launch token 전달과 HTTP/WS 인증이 확정되었다.
- ☒ sequence, replay, gap과 full resync 규칙이 확정되었다.
- ☒ schema version, Origin/Host, log와 resource 제한이 확정되었다.
- ☒ Phase 5에서 contract generation/drift와 reconnect integration test를 구현했다.
- ☐ 부분 완료 — Phase 12의 sidecar lifecycle과 CSP/capability를 component/unit 및 local bundle 정적 검증으로 확인했다. 실제 Keychain credential packaged process smoke는 남아 있다.

2026-08-24 Phase 12 local contract 검증에서 Tauri native unit 27개, backend 전체 595개 (credential 기반 4개 safe skip), UI 264개가 통과했다. arm64 `.app/.dmg`에 packaged sidecar가 포함됐고 runtime response CSP의 `127.0.0.1:0` sentinel을 READY가 검증한 exact random port로만 교체하는 경로, main-window 최소 capability, one-shot descriptor, expected와 abnormal exit bridge를 검증했다. DMG를 read-only mount한 뒤 app/sidecar strict ad-hoc signature와 DMG checksum도 확인했다. actual Keychain credential, clean-machine 실행과 Developer ID 서명-notarization은 이 ADR의 contract 구현 증거와 구분해 Phase 12 roadmap의 남은 external release smoke로 유지한다.

10. Windows source development adapter — 2026-09-06

Session 5의 현재 사용자 범위는 Windows 11 x64에서도 `UI/pnpm desktop:dev`로 실행하는 개발 환경이다. Windows native compile/smoke는 Session 6의 별도 증거이며 unsigned installer와 PyInstaller 배포본은 현재 작업의 산출물이 아니다. macOS의 Keychain, AppKit quit guard와 FD `3/4/5/6` ABI는 보존한다. 업무 Controller와 Communication Operation은 추가하거나 변경하지 않는다.

- Windows debug shell은 repository `backend/.venv/Scripts/python.exe -I -m binance_auto_trader.sidecar`를 실행한다. Credential과 token은 argument/environment에 넣지 않고 anonymous stdin 첫 frame에만 담는다. Child environment는 Windows 시스템 경로와 애플 전용 임시 경로로 제한한다.
- Frame은 **4-byte unsigned big-endian byte length + UTF-8 JSON object**이다. 전역 상한 1 MiB 안에서 BOOTSTRAP은 16 KiB, 내부 기존 configuration은 8 KiB, READY는 4 KiB, control은 256 bytes로 제한한다. 최초 parent frame은 exact `{type: "BOOTSTRAP", token, configuration}`이다. Configuration은 기존 schema와 exact read-only 필드를 그대로 사용하며 duplicate field, nonfinite JSON, 잘못된 타입과 미완성 frame을 거부한다.
- 최초 child frame은 기존 READY descriptor의 `port, session_id, runtime_pid, process_start_id, schema_version`만 포함한다. READY stream의 partial prefix/body는 native timeout 후에도 보존하며 공통 late-ready recovery가 이어 읽는다. 일반 stdout과 credential은 protocol output에 포함하지 않는다.
- 이후 parent control은 `{type: "CLOSED_ACK"}`다. HTTP shutdown으로 실제 CLOSED에 도달하고 응답을 flush한 뒤의 ACK만 process 종료로 허용한다. early ACK는 버린다. stdin EOF 또는 잘못된 control은 기존 FD5 EOF처럼 ORPHANED를 기록하고 신규 BUY를 잠그며 listener와 lifetime lock을 유지한다.
- Credential Manager는 current user의 generic target `com.binance-auto.trader.testnet/api-key`와 `com.binance-auto.trader.testnet/api-secret`를 사용한다. Blob은 trim하지 않은 printable ASCII 1~512 bytes다. PowerShell hidden prompt와 C# native bridge는 set/check/delete를 제공하고, canary는 별도 고정 target만 사용한다. Native secret buffer는 해제 전 zeroize한다. Python의 immutable 문자열까지 물리적으로 zeroize했다고 주장하지 않으며 참조 수명을 bootstrap/runtime으로 제한한다.
- App-data는 환경변수 대신 current-user `FOLDERID_LocalAppData/com.binance-auto.trader`로 결정한다. Root부터 ancestor handle을 `FILE_FLAG_OPEN_REPARSE_POINT`, delete sharing 없이 고정하고 reparse point를 거부한다. Artifact는 regular/single-link 파일이며 Rust-Python 모두 offset 0, length 1의 nonblocking exclusive `LockFileEx`를 사용한다. Liveness는 `OpenProcess/WaitForSingleObject`로 확인하고 접근 거부나 미확정 상태를 stale로 해제하지 않는다.
- Windows file durability는 writable file의 `FlushFileBuffers`를 명시적으로 사용한다. POSIX directory fsync의 동일한 crash 보장을 native 증거 없이 주장하지 않는다. Native filesystem/crash 검증은 Session 6에 남는다.
- 개발 Origin은 exact `http://127.0.0.1:5173` 한 개다. Packaged Windows Origin은 native에서 확인하지 않았으므로 allowlist에 추측 값을 추가하지 않고 Windows release startup/build를 차단한다. 기본 CSP와 capability는 확장하지 않는다. Tauri CLI는 platform configuration 뒤 `--config`를 병합하므로 launcher는 Windows override를 마지막에 선택한다.

Platform API 근거는 [Microsoft Credential 구조](#), [LockFileEx](#), [SHGetKnownFolderPath](#), [FlushFileBuffers](#), [Tauri configuration](#)을 사용한다. Binance endpoint·인증·주문 의미는 이번 변경의 대상이 아니므로 새로운 Binance 동작을 추측해서 추가하지 않았다.